Informe preliminar desafío 1

Luis Andres Castillo Chicaiza
Facultad de Ingeniería
Universidad de Antioquia
Medellín, Colombia
luis.castillo1@udea.edu.co

Marcos Castrillón Velásquez
Facultad de Ingeniería
Universidad de Antioquia
Medellín, Colombia
marcos.castrillon@udea.edu.co

I. Contextualización

El desafio dos, o sea"UdeAWorldCup", consiste en el desarrollo de un sistema de software para gestionar datos y simular el desarrollo de la Copa Mundial de la FIFA 2026. El mundial de fútbol real es un evento deportivo de alta complejidad en cuanto a la lógica en el que participan 48 selecciones nacionales. El objetivo principal es crear un programa que represente las entidades y acciones de interés de este entorno, utilizando el paradigma de Programación Orientada a Objetos (POO).

El simulador deberá gestionar el torneo, comenzando por una etapa de carga de datos que tomará las estadísticas de los equipos y fabricará de manera artificial las plantillas de 26 jugadores para cada selección. A partir de allí, el sistema deberá automatizar la conformación de 12 grupos mediante un proceso de sorteo regulado por restricciones geográficas y de ranking.

Finalmente, la plataforma gestionará el calendario de juego y simulará la totalidad de los 104 partidos del torneo hasta determinar las estadísticas finales y el equipo campeón del mundo.

II. Análisis del problema

Para el desarrollo de *UdeAWorldCup 2026*, se identificaron los principales requisitos funcionales y las restricciones técnicas planteadas por el cliente, los cuales guían el diseño e implementación del sistema.

A. Requisitos funcionales

- **Gestión de datos:** El código debe ser capaz de leer la información histórica de los 48 equipos clasificados a partir de un archivo CSV proporcionado. Con estos datos, se inicializarán estadísticas simuladas para los 26 jugadores de cada selección. Además, esta información deberá actualizarse después de cada partido y almacenarse en un nuevo archivo definido por los desarrolladores.

- **Sorteos y fase de grupos:** El programa debe organizar a las selecciones en 4 bombos según el ranking de la FIFA (ubicando al país anfitrión en el bombo 1). A partir de esto, se conformarán 12 grupos de 4 equipos elegidos de forma aleatoria, asegurando que no coincidan equipos de la misma confederación.

- **Simulación de partidos:** El sistema debe simular los 104 partidos del torneo utilizando un modelo matemático basado en "goles esperados". Adicionalmente, en cada partido se seleccionarán aleatoriamente 11 jugadores por equipo, a quienes se les asignarán estadísticas como goles, tarjetas (amarillas y rojas) y faltas, siguiendo probabilidades previamente definidas.

- **Eliminatorias y desempates:** Una vez finalizada la fase de grupos, el sistema generará la tabla de posiciones. La clasificación a octavos de final se realizará primero por puntos y, en caso de empate, se aplicarán criterios como diferencia de goles, goles a favor y un sorteo. En las fases eliminatorias, si un partido termina en empate, se utilizará el ranking FIFA para definir al ganador, considerando además un tiempo total de juego de 120 minutos.

- **Rendimiento:** El sistema debe generar estadísticas finales del torneo, incluyendo los 4 mejores equipos, los máximos goleadores y la representación de cada confederación en las etapas finales. Además, durante la ejecución, el programa debe registrar la cantidad de iteraciones realizadas y el consumo de memoria dinámica.

B. Restricciones técnicas

- **Lenguaje:** La solución debe desarrollarse utilizando Programación Orientada a Objetos (POO), modular basado en múltiples archivos. No está permitido el uso de sintaxis propia de C.

- **Restricción de librerías estándar:** No se permite el uso de la STL, ni de *structs*, ni de librerías externas (con excepción de la librería de números aleatorios).

- **Estructuras de datos personalizadas:** Todas las colecciones de datos (como listas de jugadores o calendarios de partidos) deben implementarse mediante estructuras propias, haciendo uso de punteros y memoria

dinámica como base.

- **Uso de plantillas y sobrecarga de operadores:** Se requiere aplicar conceptos avanzados de POO. Para ello:

  - Se implementará una clase plantilla `Lista<T>` para manejar estructuras dinámicas. En esta clase se sobrecargará el operador (=) para asegurar una correcta gestión de memoria.

  - Se desarrollará una clase `EntradaTabla`, en la cual se sobrecargará el operador (>) para facilitar la comparación entre equipos según los criterios de desempate.

III. Diagrama de clases

A continuación se muestra el diagrama de clases presentado para este desafio:

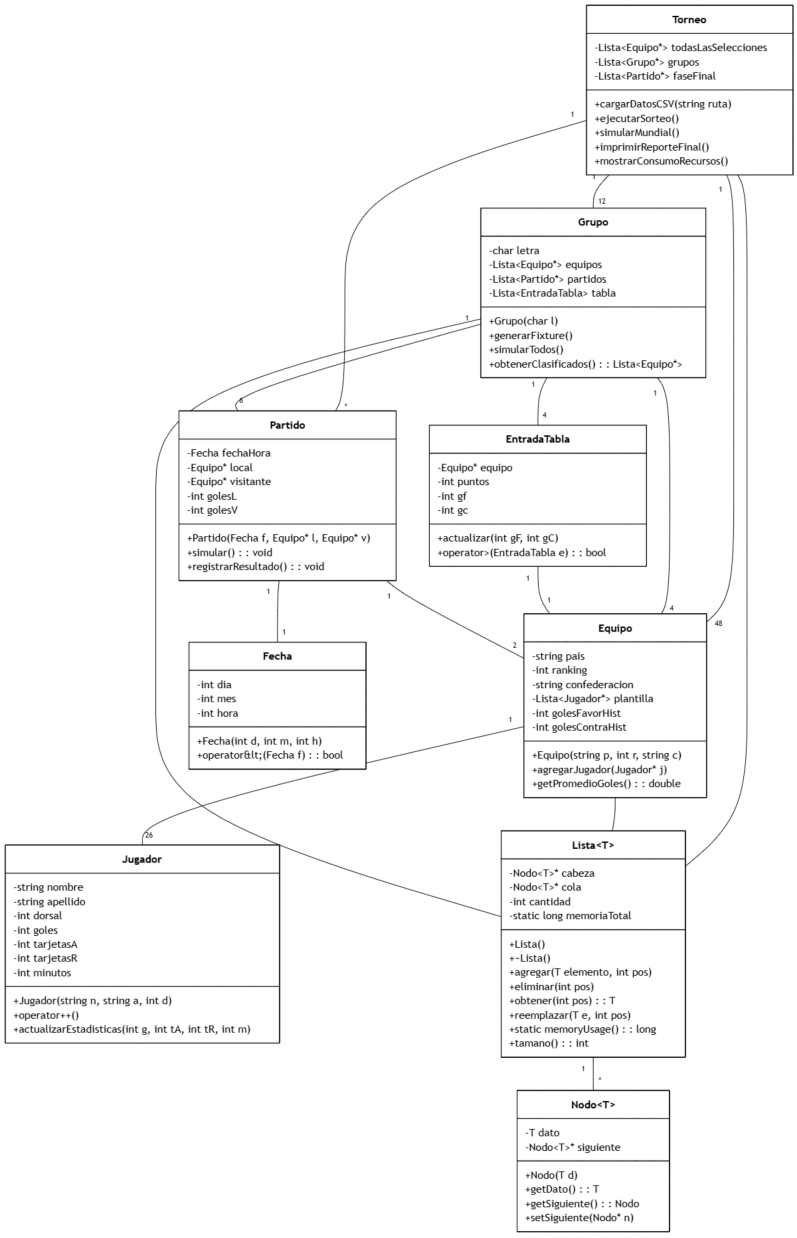

Fig. 1. Diagrama de clases